

CSC317 Computer Graphics

Tutorial 2

September 11, 2025

Assignment 1: Reminders

- Assignment 1 due **tomorrow**: Sep 12, 2025 before 11:59 pm
- Questions and Assistance
 - “Issues” page on github for Assignment 1
 - Email the TAs: csc317tas@cs.toronto.edu
- Office Hour via Zoom: Sep 12, 1:00pm - 3:00pm

Assignment 2

- Assignment 2 Due Date: September 19, 2025 before 11:59 pm
- Questions and Assistance
 - “Issues” page on github for Assignment 2
 - Email the TAs: csc317tas@cs.toronto.edu
- Office Hour via [Zoom](#): September 19, 11:00 am - 1:00 pm

Task 1: src/write_ppm.cpp

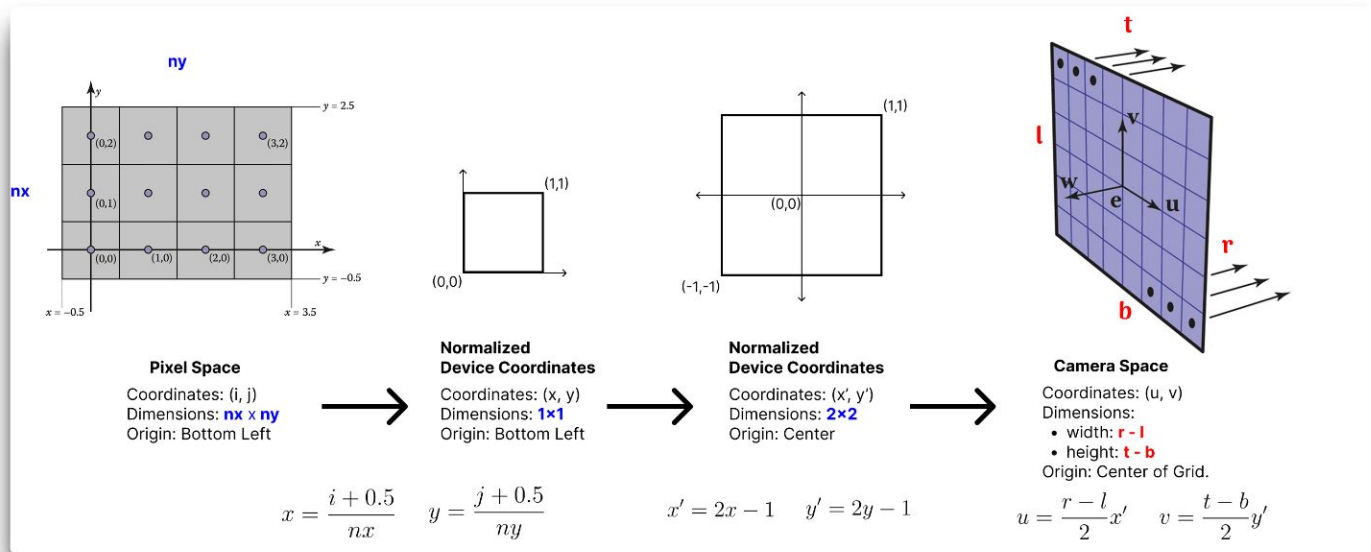
- Same as Assignment 1
- You can reuse your code

Task 2: src/viewing_ray.cpp

- Output: ray.origin, ray.direction (in worldspace)
- Input:
 - (i, j): pixel index of the pixel we want to shoot a ray through
 - (height, width): pixel size of the image
 - (camera.height, camera.width): world space (physical) size of the image
 - camera.d: focal length (we want to make sure ray.direction lands on the image plane)
 - (camera.u, camera.v): camera axes oriented in worldspace
- Ray origin is at the camera location
 - Look at Camera.h
- Careful:
 - We want the ray going through the MIDDLE of the pixel (not the top left corner), need to have an offset

Mapping 2D pixel grid to 3D image plane

How does each pixel (i, j) map to a point (u, v) on the image plane?



$$u = \frac{width}{nx}(i + 0.5) - \frac{width}{2}$$

$$v = \frac{height}{ny}(j + 0.5) - \frac{height}{2}$$

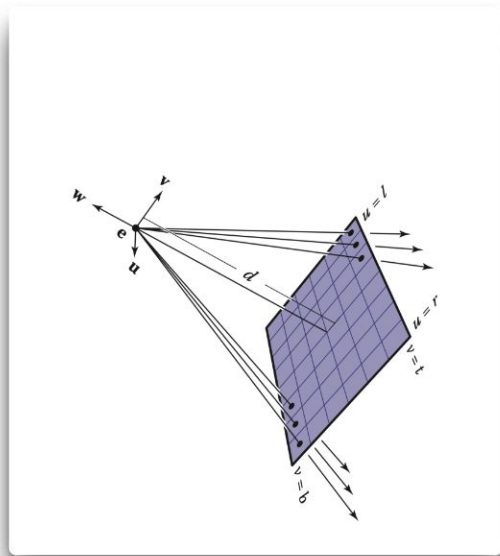
Ray Equation in Camera Space

General form:

$$\vec{p}(t) = \vec{e} + t(u(i)\vec{u} + v(j)\vec{v} - d\vec{w})$$

$$\vec{p}(t) = \vec{e} + t \begin{bmatrix} \vec{u} & \vec{v} & \vec{w} \end{bmatrix} \begin{bmatrix} u(i) \\ v(j) \\ -d \end{bmatrix}$$

$\begin{bmatrix} \vec{u} & \vec{v} & \vec{w} \end{bmatrix}$ Is a camera matrix



Task 3: src/first_hit.cpp

- Output:
 - hit_id: index of object first hit
 - t: parametric distance of the ray hit (i.e. $\text{ray.origin} + t * \text{ray.direction}$ gives the surface hit)
 - n: surface normal at the location of first hit
 - return bool: if the ray hit any object (false if not)
- Input:
 - ray: ray to search
 - min_t: minimum value of t to accept (culls surfaces behind and too close to the camera)
 - objects: list of objects in scene
- Loop through all the objects, call the object intersect with the ray, and see which object gives the smallest (greater than min_t) t value
- Note: no actual intersection checks should be done here, just call the object intersects and compare

Task 4: src/Sphere.cpp

- Test if a point q in the ray is on the sphere:
 - q : point, c : center, r : radius

$$(\mathbf{q} - \mathbf{c})^2 - r^2 = 0$$

$$(\mathbf{e} + t\vec{\mathbf{d}} - \mathbf{c}) \cdot (\mathbf{e} + t\vec{\mathbf{d}} - \mathbf{c}) - r^2 = 0$$

$$(\vec{\mathbf{d}} \cdot \vec{\mathbf{d}})t^2 + 2\vec{\mathbf{d}} \cdot (\mathbf{e} - \mathbf{c})t + (\mathbf{e} - \mathbf{c}) \cdot (\mathbf{e} - \mathbf{c}) - r^2 = 0$$

In this way we have rearranged the equation in the form: $at^2 + bt + c = 0$

Quadratic formula:
$$t = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Discriminant:
$$b^2 - 4ac$$

Task 4: src/Sphere.cpp

- Looking at the discriminant, we can determine if we have an intersection:
 - < 0 : no intersection
 - $= 0$: one intersection
 - > 0 : two intersections
- Note: with two hits, you have three cases:
 - Check both t against min_t
 - both $> \text{min_t}$: both in front
 - one $> \text{min_t}$: one in front
 - none $> \text{min_t}$: none in front
- Normal:
 - $$\frac{\mathbf{hit} - \mathbf{c}}{r}$$

Task 5: src/Plane.cpp

- Test if a point q in the ray is on the plane:
 - q : point, p_0 : point on the plane, n : plane normal

$$\vec{n} \cdot (q - p_0) = 0$$

$$\vec{n} \cdot ((\mathbf{e} + t\vec{d}) - p_0) = 0$$

$$t = \frac{-\vec{n} \cdot (\mathbf{e} - p_0)}{\vec{n} \cdot \vec{d}}$$

- Note: if $\vec{n} \cdot \vec{d} = 0$ this is undefined!
 - Either no hit, or the camera is in the plane
- Normal:
 - Plane normal (normalize it)

Task 6: src/Triangle.cpp

- Test if a point q in the ray is on the triangle:
 - With 3 corners, we can form two vectors t_1, t_2

$$\mathbf{q} = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2$$

$$\mathbf{e} + t \vec{\mathbf{d}} = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2$$

$$\mathbf{e} = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2 - t \vec{\mathbf{d}}$$

$$\begin{bmatrix} \mathbf{t}_1 & \mathbf{t}_2 & -\mathbf{d} \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \\ t \end{bmatrix} = \mathbf{e}$$

We have rearranged the equation in the form $Ax = b$

Cramer's Rule!

Task 6: src/Triangle.cpp

- Then, make sure the point lies within the triangle:

- $\alpha \geq 0$

- $\beta \geq 0$

- $\alpha + \beta \leq 1$

- Also, check if $t > \text{min_t}$

- Normal:
$$\frac{\mathbf{t}_1 \times \mathbf{t}_2}{\|\mathbf{t}_1 \times \mathbf{t}_2\|}$$

Task 7: src/TriangleSoup.cpp

- Just call first hit on the list of triangles.